

in real-time. Rainbow tables are not much in use these days, while high performance computers can be used to carry out high speed dictionary attacks. Salting is actually the coding technique required to help defeat rainbow table attacks. With a few resources, attackers can create a GPU cracking rig that can execute around 25 billion attempts or even more per second —against the stored passwords. So it's recommended that you combine three major techniques, i.e., usage of the one-way algorithm, a salt, and an algorithm that is purposely slow, to prevent GPU cracking rigs and other similar resources from exposing passwords. Some of the algorithms like PBKDF2 and SCRYPT are amazing examples of algorithms that can be used safely to store passwords.

Contextual output encoding XSS defence: Cross-site scripting (also called XSS) or JavaScript injection can be used for site defacement, session hijacking, network scanning, site redirection/phishing, undermining cross-site request forgery (CSRF) defences, data theft and keystroke logging. Contextual output encoding is one of the crucial programming techniques required to stop XSS. This is usually performed on the output (while building a user interface), at the last moment, before any untrusted data is added to the HTML part during runtime.

Content security policy: A content security policy is one of the emerging browser standards. The main reason behind this policy is to create a standardised framework in order to provide browser-based protection that stops XSS, with less developer involvement. In order to put the content security policy into practice, all JavaScript that is embedded in HTML has to be removed and then deployed in a separate external JavaScript file. If a content security policy-aware browser detects any JavaScript embedded in a HTML document – for instance, if a hacker tries to insert JavaScript — then the action is rejected. This ultimately locks down the browser, preventing many forms of XSS attacks.

Cross-site request forgery: If a user has logged into a secure site and then tries to open another tab and launch a

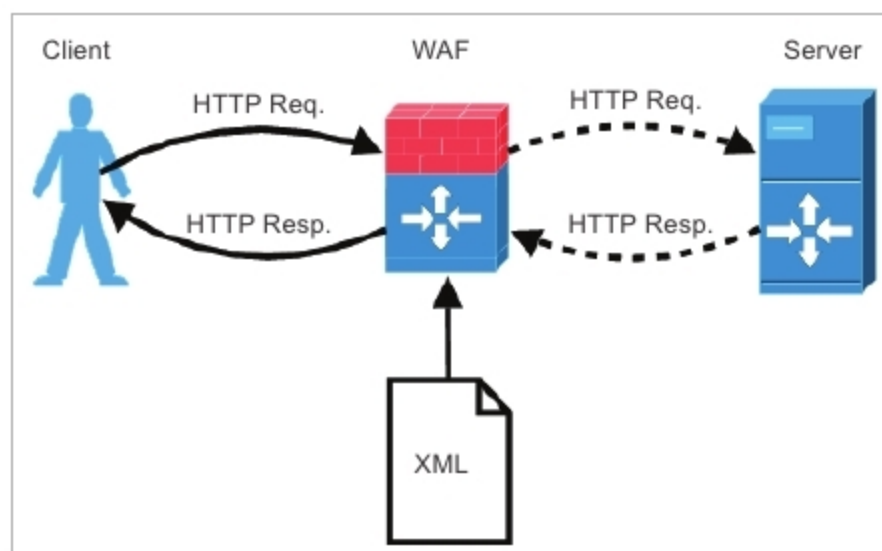


Figure 2: Architecture for a Web app firewall (Image source: *googleimages.com*)

malicious site, the secure site could possibly host a forged request taking advantage of the fact that the current user is already logged in. The malicious website could then submit a fake request, which really has a great chance of spreading harm such as tricking the user transferring money from a banking website. To prevent such attacks, Web app developers need to consider the deployment of cryptographic tokens and mandate users to re-authenticate in order to complete a transaction or event, preventing in-session hijacks.

Multi-factor authentication: Most of us know that passwords are no more the only authentication factor nowadays. A better authentication solution is 2FA (two-factor authentication) or even multi-factor authentication (MFA). Both have really been slow to gain popularity in the past due to the difficulty of always carrying around a second device just to verify the identity of a user. But as mobile devices have become a device that we always carry, it has solved this difficulty to a great extent. While native apps and SMS for MFA are not perfect, they do really reduce risks associated with password-only authentication to a great extent. Many of the consumer websites and online services are now offering multi-factor authentication, for instance Apple, Google, Facebook, Twitter, and more.

Forgotten password security design: Many of the websites allow their users to request an existing or even new password be sent to the registered email address, with little verification. But there can be a stronger process for this such as adequately validating the identity with security questions, sending the user a randomly generated token through an out-of-band method, allowing verification code in the same Web session and enforcing a lockout policy. For hackers to circumvent these kinds of security measures would really need a lot of pieces to rebuild the system, which does not make things easy for them.

Role of the Web app firewall in handling security

Web app firewalls are hardware and software solutions that protect apps. These solutions are specifically designed to examine all the incoming traffic to block malicious attack attempts, thereby compensating for any of the code sanitisation deficiencies. By securing application data from manipulation and theft, Web app firewall deployments meet one of the key criteria for PCI DSS certification. As per Requirement 6.6 of this certification, all debit and credit cardholder data held in any database must be protected. Generally, deployment of a Web app firewall doesn't require making any changes to a Web app, as it's placed well ahead of its DMZ at the edge of a network. From there, it starts acting as a gateway for all the incoming traffic, blocking any malicious request